



MangaMotion

Manga PDF → Panel-by-Panel Narrated Video

Product Requirements Document • v1.0

0. Product Overview

MangaMotion is a local-first, open-source web application that converts manga PDF files into narrated, panel-by-panel cinematic videos. A user uploads a manga PDF, the system automatically detects and crops each panel, uses AI vision to understand the scene, generates a narration script, synthesises voiceover audio, and assembles a final MP4 video — all with minimal manual intervention.

0.1 Goals

- 100% free/open-source stack (no paid APIs required to run)
- Single PDF upload → MP4 download in one workflow
- Clean, minimal browser-based UI (Gradio or Streamlit)
- Runs fully locally on CPU (GPU optional for speed)
- Handles typical manga conventions: right-to-left reading, speech bubbles, double-page spreads

0.2 Non-Goals (v1)

- Live streaming or real-time preview
- Automatic anime-style lip-sync
- Multi-user / cloud SaaS deployment
- Automatic subtitle burn-in (planned for v2)

1. System Architecture

The application is a single Python process with a browser UI. All processing is done as a sequential pipeline of five stages, each implemented as an independent module so it can be swapped or improved separately.

1.1 Pipeline Overview

Stage	Input	Output	Key Library
1 — PDF Extract	manga.pdf	page_001.png ... page_N.png	PyMuPDF (fitz)
2 — Panel Detect	page PNGs	panel_001.png ... panel_M.png (ordered)	OpenCV + kumiko
3 — Scene Analyse	panel PNGs	panels.json (descriptions)	LLaVA or moondream2
4 — Script + TTS	panels.json	panel_001.mp3 ... panel_M.mp3	Ollama + Coqui TTS
5 — Video Assemble	panel PNGs + mp3s	output.mp4	MoviePy + FFmpeg

1.2 Directory Structure

```
manga-motion/
├── app.py                # Gradio or Streamlit entry point
├── config.yaml           # All tuneable settings
├── pipeline/
│   ├── __init__.py
│   ├── extract.py        # Stage 1
│   ├── detect_panels.py  # Stage 2
│   ├── analyze.py        # Stage 3
│   ├── narrate.py        # Stage 4
│   └── assemble.py       # Stage 5
├── models/               # Downloaded model weights
├── work/                 # Temporary per-job files
│   └── <job_id>/
│       ├── pages/        # Extracted page PNGs
│       ├── panels/       # Cropped panel PNGs
│       ├── audio/        # Per-panel MP3 files
│       └── video/        # Final MP4
├── requirements.txt
└── README.md
```


2. Stage 1 — PDF Extraction

2.1 Objective

Convert each page of the uploaded manga PDF into a high-resolution PNG image that is ready for panel detection.

2.2 Library: PyMuPDF (fitz)

- Package: `pip install pymupdf`
- Pure Python, no external dependencies, very fast
- Supports DRM-free PDFs; will raise an error for DRM-protected files

2.3 Implementation Detail

Extract at 2x DPI scaling ($150 \text{ DPI base} \times 2 = 300 \text{ DPI effective}$) to preserve fine line art and text in speech bubbles. Save as PNG (lossless). Name files with zero-padded page numbers for correct sort order.

Key configuration knobs (config.yaml):

- `dpi_scale: 2` — multiplier over 150 DPI base (increase for higher quality, slower)
- `colorspace: rgb` — use 'gray' to halve file size for black-and-white manga
- `output_format: png` — or 'jpeg' for smaller files (lossy)

2.4 Pseudocode

```
import fitz # PyMuPDF
doc = fitz.open('manga.pdf')
for i, page in enumerate(doc):
    mat = fitz.Matrix(2, 2) # 2x scale
    pix = page.get_pixmap(matrix=mat, colorspace=fitz.csRGB)
    pix.save(f'pages/page_{i:04d}.png')
```

2.5 Edge Cases

- Double-page spreads: detect pages with aspect ratio > 1.5 — flag them and process as a single wide image
- Rotated pages: check `page.rotation` and auto-rotate via `fitz.Matrix.rotate()`
- Password-protected PDFs: prompt user for password in the UI; pass to `fitz.open(password=...)`

3. Stage 2 — Panel Detection & Cropping

3.1 Objective

Locate and crop individual manga panels from each page image, and sort them in the correct reading order (right-to-left, top-to-bottom for Japanese manga; left-to-right for Western).

⚠ This is the most technically challenging stage. A two-tier approach is recommended: try kumiko first (manga-aware), fall back to OpenCV contours if kumiko finds nothing.

3.2 Primary: kumiko

- GitHub: [njean42/kumiko](#) — purpose-built manga panel detector
- Install: `pip install kumiko`
- Takes a page image, returns bounding boxes of each panel
- Handles non-rectangular and borderless panels better than raw OpenCV
- Returns panels already sorted in reading-order (configurable for RTL)

3.3 Fallback: OpenCV Contour Detection

If kumiko returns zero panels (e.g. full-bleed pages or splash pages), use OpenCV:

- Convert to grayscale → binary threshold → find external contours
- Filter by minimum area (e.g., > 5% of page area) to remove gutters and noise
- Sort bounding boxes: top-to-bottom first, then right-to-left within each row
- Pad each crop by ~10px to avoid clipping panel borders

3.4 Reading Order Algorithm

Manga reading order is: right column top-to-bottom, then left column top-to-bottom. The algorithm:

1. Sort all detected bounding boxes by top-edge Y coordinate
2. Group boxes into 'rows' where Y-overlap > 50%
3. Within each row, sort boxes right-to-left (descending X)
4. Flatten the result into a single ordered list

3.5 Configuration Knobs

- `reading_direction`: rtl | ltr — toggles reading order
- `min_panel_area_pct`: 0.05 — ignore panels smaller than 5% of page

- `panel_padding_px`: 10 — extra pixels added around each crop
- `splash_page_threshold`: 1 — if only 1 panel found, treat entire page as one panel

3.6 Output

Numbered PNG files: `panel_0001.png`, `panel_0002.png` ... with a manifest JSON file recording which page each panel came from and its bounding box coordinates.

4. Stage 3 — AI Scene Analysis

4.1 Objective

For each panel image, generate a structured description including: characters, their emotions, action happening, any speech bubble text visible, and the overall mood. This becomes the input for narration generation.

4.2 Primary: Local Vision LLM via Ollama

Ollama makes it trivial to run vision-capable LLMs locally, free of charge.

- Install: `curl https://ollama.ai/install.sh | sh`
- Recommended models (in order of quality vs. speed):
 - `llava:34b` — highest quality, needs ~20GB VRAM
 - `llava:13b` — good balance, needs ~8GB VRAM
 - `moondream2` — tiny (1.8B), CPU-friendly, acceptable quality
 - `llava:7b` — good CPU performance
- Run: `ollama run moondream2` or `ollama run llava:7b`
- API: `http://localhost:11434/api/generate` (OpenAI-compatible)

4.3 Prompt Design

Send each panel image as base64 with this system prompt:

```
You are a manga scene analyst. Given a manga panel image, return ONLY
valid JSON with these fields:
{
  'characters': ['list of characters visible'],
  'action': 'what is happening in the panel',
  'emotion': 'dominant emotion/tone of the scene',
  'dialogue': 'text visible in speech bubbles (empty string if none)',
  'mood': 'one of: tense, calm, comedic, dramatic, romantic, action',
  'is_splash': true/false // full-page dramatic panel
}
```

4.4 Output: panels.json

A single JSON file is written after all panels are processed:

```
[
  {
    'panel_id': 'panel_0001',
```

```
'image_path': 'panels/panel_0001.png',
'characters': ['Naruto', 'Sasuke'],
'action': 'Naruto charges with a Rasengan',
'emotion': 'intense determination',
'dialogue': 'I will never give up!',
'mood': 'action',
'is_splash': false
},
...
]
```

4.5 Fallback: BLIP-2 (Transformers)

If Ollama is not available, use the Hugging Face transformers library with BLIP-2 for basic image captioning. Quality will be lower (no structured JSON output, requires post-processing the caption text).

- `pip install transformers accelerate`
- Model: Salesforce/blip2-opt-2.7b (5.5GB download)

5. Stage 4 — Script Writing & Voiceover

5.1 Objective

For each panel, generate a 1-3 sentence narrator script and synthesise it into an audio file using a local, free text-to-speech engine.

5.2 Script Generation — Local LLM via Ollama

Use the same Ollama instance (text-only model) to convert the structured panel JSON into a narrator script:

- Recommended model: mistral:7b or llama3:8b — fast, high quality
- System prompt context: previous 2 panels (for narrative continuity)

Narrator Prompt Template:

```
You are narrating a manga. Given the scene description below, write
1-3 sentences of cinematic narration. Be evocative, not just descriptive.
Match the mood. If there is dialogue, quote it naturally.
```

```
Scene: {panel_json}
Previous context: {last_2_narrations}
```

```
Write ONLY the narration text, no extra commentary.
```

5.3 Text-to-Speech — Coqui TTS (Free & Local)

- GitHub: coqui-ai/TTS — pip install TTS
- No API key, runs fully offline after model download
- Recommended model: tts_models/en/ljspeech/tacotron2-DDC (fast, good quality)
- Or: tts_models/en/vctk/vits for multi-speaker (different voices per character)

Usage:

```
from TTS.api import TTS
tts = TTS('tts_models/en/ljspeech/tacotron2-DDC')
tts.tts_to_file(text=narration, file_path='audio/panel_0001.wav')
```

5.4 Alternative TTS Options (Free)

Option	Quality	Speed	Setup
Coqui TTS	★★★★☆	Medium	pip install TTS
pyttsx3	★★☆☆☆	Very fast	pip install pyttsx3 (uses OS voice)
espeak-ng	★★☆☆☆	Very fast	apt install espeak-ng
edge-tts (MS)	★★★★★	Fast (needs internet)	pip install edge-tts
kokoro-82m	★★★★☆	Fast on CPU	pip install kokoro

 *edge-tts uses Microsoft's servers (free, no key needed) but requires an internet connection. It produces the best quality of all free options.*

5.5 Audio Post-Processing

- Convert WAV → MP3: use pydub (pip install pydub) with ffmpeg backend
- Add 0.3s silence padding at end of each clip for natural pacing
- Normalise audio levels across all clips: sox or ffmpeg loudnorm filter

6. Stage 5 — Video Assembly

6.1 Objective

Combine each panel image with its voiceover audio into a clip, add cinematic effects, optional background music, and stitch all clips into a final MP4 video.

6.2 Library: MoviePy

- `pip install moviepy` — pure Python video editor wrapping FFmpeg
- Each panel becomes a clip lasting exactly as long as its audio
- Clips are concatenated in panel order

6.3 Ken Burns Effect (Cinematic Pan & Zoom)

The Ken Burns effect makes static images feel alive. For each panel:

5. Randomly choose an effect type: zoom-in, zoom-out, pan-left, pan-right, or diagonal
6. Start frame: original crop at 100% size (or slight zoom)
7. End frame: zoomed to 115-120% or panned to opposite edge
8. Interpolate linearly across the clip duration
9. For dramatic (`is_splash=true`) panels, use slower zoom and longer hold

Ken Burns Pseudocode:

```
def ken_burns(image, duration, zoom_start=1.0, zoom_end=1.15):
    frames = []
    for t in linspace(0, duration, fps * duration):
        scale = lerp(zoom_start, zoom_end, t / duration)
        # crop centre at current scale, resize to output resolution
        frames.append(crop_and_scale(image, scale))
    return ImageSequenceClip(frames, fps=fps)
```

6.4 Sound Design (Optional but Recommended)

Layer	Source	Notes
Voiceover	Stage 4 output	Primary audio, always on
Background Music	Free Music Archive / Pixabay	Loop ambient track, duck during VO
Scene SFX	Freesound.org (CC0 sounds)	Based on mood field: action → impact SFX

Layer	Source	Notes
Page-turn sound	Any CC0 whoosh sound	Play between panels

Use pydub's `overlay()` to mix audio layers. Lower background music to -15dB under narration.

6.5 Dialogue Text Overlays

For panels where dialogue is detected, render the dialogue text as a subtitle-style overlay:

- Use MoviePy's TextClip with a manga-style font (e.g., Bangers, CC Wild Words)
- Position at bottom of frame with semi-transparent black background
- Display for the duration of the panel clip
- Use Japanese-style vertical text layout if the dialogue is in Japanese

6.6 Final Export Settings

- Resolution: 1920×1080 (upscale panels using Lanczos; add black bars for non-16:9)
- Frame rate: 24 fps
- Video codec: H.264 (libx264) — universally compatible
- Audio codec: AAC at 192kbps
- Preset: medium (balance of speed vs. file size)

FFmpeg command (final render):

```
ffmpeg -i panels_concat.mp4 -i mixed_audio.mp3 \  
-c:v libx264 -preset medium -crf 20 \  
-c:a aac -b:a 192k \  
-movflags +faststart \  
output_final.mp4
```

7. Web UI (Gradio)

7.1 Framework Choice: Gradio

- pip install gradio — single Python file, instant web UI
- Built-in file upload, progress bars, and video preview widget
- Runs on `http://localhost:7860` by default
- Alternative: Streamlit (pip install streamlit) — more control, more code

7.2 UI Layout

Left Panel — Input & Settings:

- File uploader (accepts .pdf only)
- Reading direction toggle: Japanese RTL / Western LTR
- TTS Voice selector dropdown
- Background music toggle + volume slider
- Output resolution: 720p / 1080p
- Button: "Generate Video"

Right Panel — Output & Progress:

- Stage progress bar (Stage 1/5 ... 5/5)
- Current panel preview (updates in real time)
- Log console (collapsible)
- Video player (shown when complete)
- Download button: "Download MP4"

7.3 Gradio Pseudocode

```
import gradio as gr

def run_pipeline(pdf_file, direction, voice, music, resolution,
progress=gr.Progress()):
    job_id = uuid4()
    progress(0.0, desc='Extracting pages...')
    pages = extract_pages(pdf_file, job_id)
    progress(0.2, desc='Detecting panels...')
    panels = detect_panels(pages, direction)
    progress(0.4, desc='Analysing scenes...')
    scene_data = analyse_panels(panels)
    progress(0.6, desc='Generating narration...')
```

```
    audio_files = generate_narration(scene_data, voice)
    progress(0.8, desc='Assembling video...')
    output_path = assemble_video(panels, audio_files, scene_data, music,
resolution)
    progress(1.0, desc='Done!')
    return output_path

with gr.Blocks(title='MangaMotion') as app:
    # ... UI layout ...
    btn.click(run_pipeline, inputs=[...], outputs=[video, download])

app.launch()
```


8. Complete Technology Stack

Component	Library / Tool	Install Command	License
PDF Extraction	PyMuPDF	pip install pymupdf	AGPL-3.0
Panel Detection	kumiko	pip install kumiko	EUPL-1.2
Image Processing	OpenCV	pip install opencv-python	Apache-2.0
Vision AI	Ollama + LLaVA/moondream	curl ollama.ai/install.sh sh	MIT
Script Writing	Ollama + Mistral/Llama3	(same Ollama instance)	MIT / Apache
Text-to-Speech	Coqui TTS	pip install TTS	MPL-2.0
TTS Alternative	edge-tts	pip install edge-tts	MIT
Video Editing	MoviePy	pip install moviepy	MIT
Video Encoding	FFmpeg	apt install ffmpeg	LGPL-2.1
Audio Processing	pydub	pip install pydub	MIT
Web UI	Gradio	pip install gradio	Apache-2.0
Async/Jobs	Python threading / queue	(stdlib)	PSF

9. Detailed Build Order

Claude Code should build the application in this exact order to allow incremental testing at each step:

Phase 1 — Foundation (Stages 1 + 2)

10. Scaffold project: create directory structure, config.yaml, requirements.txt, __init__.py files
11. Implement extract.py (Stage 1): PDF → page PNGs; test with a sample manga PDF
12. Implement detect_panels.py (Stage 2): kumiko + OpenCV fallback; visualise panel crops
13. Write a CLI test runner: `python -m pipeline.test_stages 1 2 --pdf sample.pdf`
14. Verify: inspect panel crops manually; tune `min_panel_area_pct` if needed

Phase 2 — AI Analysis (Stage 3)

15. Set up Ollama with moondream2 (smallest, CPU-safe)
16. Implement analyze.py: send panel images to Ollama, parse JSON response
17. Test on 10 panels; inspect panels.json output quality
18. Tune prompt if fields are missing or hallucinated
19. Add retry logic (3 attempts) in case Ollama returns malformed JSON

Phase 3 — Audio (Stage 4)

20. Install Coqui TTS and download default model (first run auto-downloads)
21. Implement narrate.py: LLM script generation + TTS synthesis
22. Test with 5 panels; listen to audio output quality
23. Add pydub audio normalisation and silence padding
24. Wire edge-tts as optional fallback (config flag: `tts_engine: coqui | edge | pyttsx3`)

Phase 4 — Video (Stage 5)

25. Implement assemble.py: basic static panel + audio → per-clip MP4
26. Test concatenation: stitch 5 clips into one MP4
27. Add Ken Burns effect
28. Add background music layer with volume ducking
29. Add dialogue text overlays
30. Final export at 1080p H.264

Phase 5 — UI

31. Build Gradio app.py with full input panel and output panel
32. Add real-time progress updates using `gr.Progress()`
33. Add live panel preview during analysis stage
34. Add video player and download button on completion
35. Test end-to-end with a full chapter (30-50 panels)
36. Add error handling: show friendly messages for Ollama not running, missing FFmpeg, etc.

10. config.yaml — All Settings

```
# MangaMotion Configuration

pdf_extraction:
  dpi_scale: 2          # 1=150dpi, 2=300dpi, 3=450dpi
  colorspace: rgb       # rgb or gray
  output_format: png    # png or jpeg

panel_detection:
  reading_direction: rtl # rtl (Japanese) or ltr (Western)
  engine: kumiko         # kumiko or opencv
  min_panel_area_pct: 0.04 # ignore panels < 4% of page
  panel_padding_px: 12
  row_overlap_threshold: 0.5

scene_analysis:
  engine: ollama
  ollama_host: http://localhost:11434
  vision_model: moondream2 # or llava:7b, llava:13b
  max_retries: 3

narration:
  script_model: mistral:7b # or llama3:8b
  context_window: 2        # how many prev panels to include
  tts_engine: coqui        # coqui, edge, or pyttsx3
  tts_model: tts_models/en/ljspeech/tacotron2-DDC
  silence_pad_ms: 300

video:
  resolution: 1920x1080
  fps: 24
  codec: libx264
  crf: 20                # 18=high quality, 28=smaller file
  ken_burns: true
  ken_burns_zoom: 1.15   # max zoom level
  background_music: false
  music_volume_db: -15   # relative to narration
  dialogue_overlay: true
  dialogue_font: Bangers
```

11. requirements.txt

```
# Core pipeline
pymupdf>=1.24.0
kumiko>=0.9.0
opencv-python>=4.9.0
numpy>=1.26.0
Pillow>=10.3.0

# AI / LLM
ollama>=0.2.0          # Python client for Ollama
requests>=2.31.0       # fallback HTTP calls to Ollama

# TTS
TTS>=0.22.0            # Coqui TTS
edge-tts>=6.1.9        # Microsoft Edge TTS (optional)
pydub>=0.25.1

# Video
moviepy>=1.0.3

# UI
gradio>=4.31.0

# Utils
pyyaml>=6.0.1
tqdm>=4.66.0
```

12. Known Challenges & Mitigations

Challenge	Risk	Mitigation
Irregular panel layouts	High	kumiko handles most cases; user can manually override panel order
LLaVA misidentifying characters	Medium	Let user provide a character name map in config; inject into prompt
Coqui TTS cold start slow	Medium	Pre-load TTS model at app startup; show model loading progress
Ollama not installed/running	High	Detect at startup; show clear install instructions in UI error banner
Very long chapters (100+ panels)	Medium	Process in background thread; show per-panel progress; stream to video incrementally
Double-page spreads	Medium	Detect wide pages (aspect > 1.6); treat as single panel; no splitting
Vertical text in speech bubbles	Low	Accept limitation in v1; add Manga-OCR integration in v2
FFmpeg not installed	High	Check at startup; provide install instructions per OS in README

13. Claude Code Prompt Template

Use the following prompt when starting a Claude Code session to build this application:

— START CLAUDE CODE PROMPT —

Build a Python application called MangaMotion following the PRD at [path/to/prd.docx].

Requirements:

- Use the exact directory structure from Section 1.2
- Implement all 5 pipeline stages as separate Python modules in pipeline/
- Each stage must be independently runnable for testing
- All settings must be read from config.yaml (use PyYAML)
- The Gradio UI must show real-time progress per stage
- Include a requirements.txt matching Section 11
- Include a README.md with setup instructions for macOS, Ubuntu, and Windows
- Add proper error handling: if Ollama is not running, show a clear message
- Include a sample manga page (public domain) in test_data/ for automated testing

Build in this order:

37. Project scaffold (directories, config.yaml, requirements.txt)
38. Stage 1: extract.py
39. Stage 2: detect_panels.py
40. Stage 3: analyze.py
41. Stage 4: narrate.py
42. Stage 5: assemble.py
43. app.py (Gradio UI)
44. README.md

After each stage, write a quick test that prints success/failure so I can verify incrementally.

— END CLAUDE CODE PROMPT —